Long-branch removal

The Python prototype and the Julia implementation: comparison, changes made, and decisions taken

Python prototype	long_branch_explorer/programs/long_branch_identifier.py, prerun_param_optimiser.py, clan_check.py
Julia implementation	Phyju/Long_Branch_Remover/long_branches_identifier.jl, long_branches_statistical_analyses.jl, long_branch_rules.jl
Supporting libraries	TreesIO, TreeUtils, TreeStats
Date	13 August 2026

This document records two things. First, how the Julia implementation differed from the Python prototype it was based on, rule by rule. Second, every change subsequently made to the Julia code, with the reasoning behind each decision, including the cases where the Julia version deliberately departs from Python.

The Python code is a prototype written with ChatGPT to test whether the idea was worth pursuing. The Julia code is the implementation intended for use. Where the two disagree, the question in each case was which behaviour is correct, not which came first.

> ***Governing principle.*** *Ambiguity resolves to retention. A taxon or clade is removed only when the evidence positively supports removal; anything undecidable, untestable or borderline is kept. Every rule below was checked against this, and the boundary cases changed where they did not comply.*

> ***Preliminary.*** *This document was produced with Claude AI during the debugging and revision of the Julia implementation, which was itself carried out with Claude AI. It describes code that is still being tested, and should be treated as a working record rather than a settled specification. Anything here may change as testing continues.*

1. What the pipeline does

Four independent rules examine each gene tree and nominate taxa for removal. The software never prunes: it writes a list of what could be pruned under each rule, and the user decides. Each rule combines a local trigger with a global safeguard, and both must be exceeded before anything is nominated.

Rule	Question it asks	Local unit
Terminal	Is this taxon's own branch long compared with the same taxon in other genes?	taxon
Named clade (stem)	Is the branch subtending this named clade long compared with the same clade in other genes?	named clade
Internal / long subtree	Is there a chain of long internal branches, and does it identify a subtree to remove?	gene tree
Quartet	Is this taxon extreme relative to its immediate neighbours in this tree?	local neighbourhood

The trigger for the first three has the form

```
trigger = (K - 1) * median_for_this_unit + global_median
```

so that a perfectly typical unit (where its own median equals the global constant) must exceed K times that constant. The quartet rule instead compares distances within a single tree and needs no cross-gene background.

2. Summary of changes made

Area	Change	Section
Quartet masking	Adopted Python's comparator rule; mask groups now derived per tree from the largest monophyletic fragment	4.4
Quartet comparators	Three comparators required, sorted by distance from the anchor; level-order anchor search	4.4
Quartet safeguard	Changed from a quantile of ratios to the global terminal branch-length quantile	4.4
Quartet comparators	Minimum comparator distance restored, and raised to the same scale in the denominator guard	4.5
Statistical pools	Set changed to Vector: every observation now counted	5.1
Global constants	All three now medians of per-unit medians rather than pooled medians	5.2
Minimum observations	Units seen in fewer than three trees are no longer testable	5.3
Dataset size guards	Cross-gene rules disabled at three trees or fewer, with warnings below 10 and 50	5.4
Internal safeguard	Was accepted and ignored; now applied at the branch subtending the clade to be removed	4.3
Internal asymmetry	Added Python's side-asymmetry and smaller-side conditions	4.3
Clade aliasing	Clade names resolving to one branch are collapsed for testing, and all record the measurement	6
Nested clades	A clade inside an already-removed clade is no longer reported separately	6
Boundary cases	All comparisons made strict, so equality retains	7
Reporting	Two clade-deletion figures and tables added	9
Defects	Five errors found and fixed	8
Libraries	Long-branch code moved out of TreeStats and TreeUtils	11

3. Defects that prevented the code from running

Five errors were found. Three would have thrown at run time; two were silent.

3.1 Exported name did not match the function

TreeUtils exported `get_outermoste_edge` while the function was `get_outermost_edge`. Because TreeStats called the correctly spelled but unexported name, the long-subtree rule threw `UndefVarError` the first time any tree contained a qualifying chain of long branches. This rule may therefore never have executed.

3.2 Integer type annotation blocked fractional K

`stats_for_terminals` declared its threshold as `Int64` while both drivers pass a `Float64`, giving a `MethodError` on the first call. The statistics script had the same problem in reverse, assigning parsed values into `Int64` locals, so any fractional K such as 3.45 raised `InexactError`. Both fixed; fractional K now works end to end.

3.3 Dead safeguard

The internal-branch retention threshold was computed in the driver, passed into `identify_long_branched_substrees`, and never referenced in the body. The second value in safe.txt had no effect whatsoever. Now applied - see 4.3.

3.4 Field name mismatch

`annotate_node!` wrote to `.other_attribute` while the field defined in TreesIO is `other_attributes`. Never called, so never triggered, but it would have errored on first use.

3.5 Missing dependency

`quartets_EA` calls `combinations` without TreeStats importing Combinatorics. A `using` statement in TreeUtils does not bring a name into TreeStats. Dependency and import both added.

3.6 Argument handling

Three further problems in the command line: `--clades` was optional but used unconditionally; the help text described three trigger values while the code indexed four; and `-p` carried both `required = true` and an unreachable default. All corrected, and the input files are now length-checked at start-up with named errors instead of a BoundsError deep in the run.

4. The four rules

4.1 Terminal branch rule

Substantively unchanged, and the one part of the pipeline the two implementations always agreed on in form. Python offers a plain ratio test by default and the additive hybrid under a flag; Julia implements only the hybrid, which is what the Python runs used. The global constant in that formula did change - see 5.2.

4.2 Named-clade stem rule

Same trigger form. Three changes: the global constant is now a median of per-clade medians (5.2); a clade measured in fewer than three trees is no longer testable (5.3); and clade names that resolve to the same physical branch are now handled together rather than independently (6).

4.3 Internal branch and long-subtree rule

This is where the two implementations differ most, by design. Python tests each internal edge independently against a fixed global ratio. Julia builds connected chains of long internal branches - stacks - and requires a chain of at least `-p` branches before acting. That is a deliberate improvement and was kept.

What was missing were Python's conditions on *which side* of the branch to remove. Julia removed the smaller side by taxon count, full stop. Python asks two further questions, and both are now implemented:

- **Asymmetry.** The median distance from the branch out to the tips is computed for each side, excluding the tested branch itself. The two must differ by more than `--internal_side_ratio`, otherwise the branch says nothing about which side is the problem and nothing is removed.
- **Direction.** The side removed is the one with the *longer* median depth - the side that is itself long-branched, not merely the side sitting beyond a long stem.
- **Size.** That side must also be strictly smaller in taxon count. If the long-looking side is the larger one, nothing is pruned; the rule does not fall back to removing the other side.

The practical effect: a small clade of short-branched taxa hanging off a long stem is no longer removed by this rule. There, the long branch is the stem, not the clade.

Where the safeguard sits

The retention threshold was applied first as a filter before chains were built. That was wrong: a modest branch in the middle of a chain would split one stack into two and push both below `-p`, so a genuine long-branch path would be missed. It now applies at the point of action, to the branch subtending the clade that would be deleted.

This mirrors Python, where a rescued edge still counts toward the downstream path-consistency tally - the rescue blocks the prune, it does not erase the branch from the evidence.

```
# chains are built on the trigger alone
stacks = find_long_branch_stacks(tree, bls_of_tree, trigger, min_stack)
for stack in stacks
    outermost = get_outermost_edge(tree, stack)
    bl_outermost > internal_branches_retention_threshold || continue   # safeguard
    taxa = clade_for_removal_by_asymmetry(tree, outermost, side_ratio) # asymmetry
    isempty(taxa) && continue
    ...
end
```

Four gates in sequence

A clade is removed by this rule only if all four pass: the chain contains at least `-p` long branches; the branch subtending the clade exceeds the global internal quantile; the two sides differ by more than `-r`; and the deeper side is also the smaller one.

4.4 Quartet rule

Three separate problems, all now corrected.

Masking was close to inverted

The intent of the mask file is to name clades that are naturally long-branched, so the quartet test does not penalise them. Python's rule:

```
def comparator_allowed(ctx, focal_name, comparator_name):
    focal_group = ctx.taxon_to_quartet_mask_group.get(focal_name)
    comp_group  = ctx.taxon_to_quartet_mask_group.get(comparator_name)
    if focal_group is None:
        return comp_group is None
    return comp_group is None or comp_group == focal_group
```

The Julia version instead excluded only the focal taxon's own clade-mates. The two differ in three cases:

Focal	Comparator	Python	Julia (before)
unmasked	in clade G	not allowed	allowed
in clade G	same clade G	allowed	not allowed
in clade G	different clade G'	not allowed	allowed

So an ordinary taxon could be compared against known long-branch taxa, inflating its local reference; while a ctenophore could not be compared against other ctenophores, which is precisely the fair comparison the mask exists to permit. Python's rule is now implemented exactly.

Mask membership is now derived per tree

Python decides membership per tree, keeping only the largest monophyletic fragment of each named clade, and only if it holds at least two taxa. Julia used a flat lookup against the definition file, so a taxon was masked everywhere regardless of where it actually sat.

The consequence of adopting Python's approach is deliberate and worth stating: a clade member displaced outside its fragment in some gene tree is no longer masked there. It loses the protection of being compared against its relatives, and so is more likely to be flagged - which is appropriate, since a displaced member of a known long-branch clade is close to the definition of the artefact being hunted.

Comparator selection

Python requires exactly three comparators, drawn one per branch at the anchor and topped up from the remainder, with tips sorted by patristic distance from the anchor. Julia accepted two, and took the first tip in breadth-first order - nearest by number of edges, not by branch length. Both now follow Python, and anchors are searched level by level.

The safeguard

Julia asked whether the quartet *ratio* was unusual compared with all other ratios. Python asks whether the taxon's own *terminal branch* is long in absolute terms. These are different questions, and Python's is the better one.

A ratio-quantile safeguard is a second threshold on the same statistic, so it merely restates K as max(K, quantile) and lets the data override the K that was chosen deliberately. It also fails on the case that matters: in a very short-branched neighbourhood, comparators at 0.001 and a focal at 0.01 give a ratio of 10 - spectacular locally, trivial in absolute terms, and high relative to other ratios too, so it survives. Long-branch attraction is driven by absolute amounts of change, so such a branch cannot generate the artefact.

The quartet rule now uses the same global terminal branch-length quantile as the terminal rule, one value serving both, as Python's `--global_rescue` does. This also subsumes the old hard-coded skip of taxa with branches below 1e-6: that was the same test with an arbitrary cutoff instead of one taken from the data.

4.5 Degenerate comparators

This one took three attempts and is worth recording in full, because the reasoning that went wrong is more instructive than the fix.

The Julia code originally excluded a tip from serving as a comparator when its own terminal branch was at or below 1e-6. I removed that during the port on the grounds that Python has no equivalent. Python indeed has none: its only protection is a test of the local median against 1e-12, which prevents division by exact zero and nothing more.

The first test run showed the cost. Quartet ratios spanned 7e-7 to 20805 against a median of 0.73, with more than fifty values above 100 - all of them degenerate distances rather than biology. Tree inference pins branches it cannot estimate at a minimum value, and IQ-TREE uses 1e-6; such a branch is *unestimated*, not short, so comparing a taxon against it is comparing against missing data.

The failed attempt. I restored the guard but changed the quantity it tests, filtering on the comparator's distance from the anchor rather than on its own terminal branch, and argued that this was the better choice because it is the quantity entering the median. It is not. In a cluster of near-identical sequences every tip sits on a floor-length branch, but two or three edges out from the anchor the accumulated distance already exceeds 1e-6 - so a distance test keeps precisely the tips that should be excluded. Measured, it left the maximum ratio unchanged at 20805 and made matters worse: excluding a few tips left some anchors with fewer than three comparators, pushing the search outward, and a more distant anchor inflates the numerator. The number of extreme values rose.

The fix. The original criterion, restored exactly:

```
const MIN_COMPARATOR_BRANCH = 1e-6

neighbour_of_tip = first(neighbor_labels(tree, node))
tip_edge         = tree[node, neighbour_of_tip]
if ismissing(tip_edge.length) || tip_edge.length <= MIN_COMPARATOR_BRANCH
    continue          # unestimated branch: no information about local scale
end
```

	no guard	distance from anchor	own terminal branch
maximum ratio	20805	20805	1300
values above 100	44	54	2
ratios computed (n)	38270	38270	38270
99th percentile	3.19	3.19	3.04
median	0.730	0.731	0.729

Note the last row of counts: *n* is unchanged throughout. Excluding those comparators cost no coverage at all - every taxon still found three valid comparators at some anchor. The artefact was removed without making any taxon untestable.

> ***Two lessons.*** *The guard was an improvement over the prototype, not a deviation from it, and removing it because Python lacked one was the wrong test to apply. And the reason the problem was visible at all is that the Julia statistics script pools the quartet ratios: Python computes the same statistic with the same weakness, never pools it, and its optimiser does not analyse quartets - so in Python the quartet K is chosen blind against a partly meaningless distribution.*

5. Statistical backgrounds

5.1 Sets became lists

All four global distributions were stored as `Set{Float64}`, which keeps only distinct values. Every global median and every retention quantile was therefore computed over the set of distinct branch lengths rather than over the observations.

This is a bias, not a rounding detail. A median describes an empirical distribution, so each observation must carry its weight. Worse, the bias has a direction: duplicate branch lengths pile up at the short end, where rounding collapses values together and minimum-length branches accumulate. Deduplicating strips that mass and leaves the long tail intact, so medians and upper quantiles move up and every threshold becomes more permissive.

It is also unpredictable. Whether two branches collapse depends on whether they are bit-identical as Float64, which depends on how many digits the tree-inference program printed - so the size of the distortion varies with output precision rather than being a consistent, correctable offset.

```
# before                                    # after
global_set_of_bls::Set{Float64}             global_set_of_bls::Vector{Float64}
union!(global_set_of_bls, Set(values))      append!(global_set_of_bls, values)
```

All four pools - terminal branches, clade stems, internal branches and quartet ratios - are now lists.

5.2 The global constant in each trigger

The additive constant sits in every trigger. When a unit is exactly typical, the formula reduces to K times that constant:

```
trigger = (K - 1) * M + M  =  K * M     when m_unit == M
```

That calibration only holds if M is the same kind of quantity as the local median - a median of per-unit medians, not a median of raw branches. Mixing the two makes the effective K drift from the K that was set, by an amount depending on how unevenly the units are sampled.

Python applies this reasoning to terminals but pools for stems, so it is not consistent with itself. The Julia version now applies it throughout, with the global term matched to whatever the local unit of that rule is:

Rule	Local unit	Global constant	One vote per
Terminal	taxon	median of per-taxon medians	taxon
Clade stem	named clade	median of per-clade medians	clade
Internal	gene tree	median of per-tree medians	gene

The internal case deserves a note. Individual internal branches have no identity across trees, but a gene tree does, and its median internal length captures that gene's overall rate. Pooling every branch would weight the constant by tree size, letting the best-sampled genes set it.

> *The safeguard quantiles remain pooled in all three cases: a percentile is meant to describe the distribution of real branches, and that is exactly what it should continue to do.*

5.3 Minimum observations

Python will not test a taxon or clade observed in fewer than three trees. Julia had no such gate and would test on a median of one or two values. Now implemented as a module constant, applied to terminals and clade stems - the two cross-gene rules. The quartet rule is within-tree and does not arise.

The mechanism is to issue no trigger at all for an under-observed unit, rather than filtering at test time, so such a unit cannot be removed by that rule. Its branch lengths still contribute to the global background: it informs the distribution, it simply cannot be judged against its own median. Both functions print what was skipped and why.

5.4 Dataset size

Python disables all cross-gene tests at three trees or fewer, and warns below 10 and below 50. Julia had no guard. Now implemented: at three trees or fewer the terminal, clade-stem and internal rules are switched off entirely and only the quartet rule runs, since it needs no cross-gene background.

6. Clade aliasing

This was the change with the largest effect on the clade rule, and the one that prompted the original comparison.

6.1 The problem

Two clade names can resolve to the same physical branch in a given tree. If CNID is split across the tree and its largest monophyletic fragment happens to be the CNID_A taxa, then in that tree CNID and CNID_A are one branch. Python collapses them; Julia treated them as independent throughout.

The consequences were threefold. The branch was tested twice, against two different cross-gene medians, so it could be called long under one name and not the other. It contributed two values to the background pool instead of one. And its taxa were written to the output twice.

6.2 What was implemented

Three separate decisions, deliberately not all the same:

Question	Rule adopted	Why
Which measurement does each clade record?	Every name that resolves to the branch records it	CNID must not lose a tree simply because CNID_A described the same branch there. This was a genuine bug in Python, where the wider clade silently loses those trees from its own median.
What enters the pooled background?	The branch, once	Otherwise the global constant is inflated by how many names happen to describe one branch.
Which threshold tests the branch?	The name with the smallest definition	That name describes the branch most tightly and its median comes from the same taxa in every tree, so it is the comparable estimate. The wider clade's median mixes full-clade stems with fragment stems.
Which name is credited in the report?	The name with the largest definition	The branch stands for that clade in this tree. Naming and calibration are different questions, so the two rules point deliberately in opposite directions.

6.3 Nested clades

Separately, Python suppresses a clade nested inside one already dropped: if CNID goes, CNID_A's taxa are already going and reporting it again is noise. Now implemented - candidates are ordered largest first, and any whose taxa are a subset of one already accepted is skipped with a printed note.

7. Conservative by default

Every removal decision was audited against the governing principle. Most already complied: undefined medians, insufficient asymmetry, exact ties, missing branch lengths and unbuildable quartets all retain. Two comparisons did not, and were made strict, so that a value sitting exactly on a threshold is kept.

Test	Comparison	At exact equality
Quartet K	local_ratio > K	retain
Quartet safeguard	terminal_bl > threshold	retain
Terminal trigger and safeguard	bl > both	retain
Stem trigger and safeguard	bl > both	retain
Internal trigger	bl > trigger	retain
Internal safeguard	bl <= threshold blocks	retain

Test	Comparison	At exact equality
Side asymmetry	ratio <= threshold blocks	retain
Smaller-side condition	size >= other blocks	retain

This is a deliberate departure from Python, which uses non-strict comparison on the quartet ratio. The practical effect is small, since exact floating-point equality is rare, but the direction is now consistent throughout.

8. Deliberate differences retained

Not every difference was a defect. The following are decisions where the Julia implementation departs from the prototype on purpose.

Difference	Reasoning
Chains of long internal branches (`-p`) replace Python's downstream path-consistency fraction	Requiring several stacked long branches is a clearer and stronger statement about a long-branch path than a fraction computed over downstream edges.
Named-clade stems remain in the internal-branch distribution	An internal branch is an internal branch. There is nothing remarkable about the branch subtending a clade that happens to have a name, and no natural way to carve internal branches into groups.
A clade may be reported by both the stem and the internal rule	The software reports rather than prunes. The clade rule is partly a judgement, since it depends on believing a named clade is long; a user who distrusts it can take only the internal-rule lines. Being told a taxon was nominated twice is information, not duplication.
No per-taxon diagnostic CSVs	Those existed in the prototype to work out what the rules should be. That question is now settled.
No FLAG category	The output file lists what might be pruned. A taxon that could not be tested is not a candidate for pruning, so it does not belong there. Coverage is reported separately instead.
ClanCheck tests unrooted clans and does not prune	Python's version tests clades on the rooted input tree, which is not the same question.
Tips whose own terminal branch is at or below 1e-6 cannot serve as comparators	A branch at the inference floor is unestimated, not short. Python has no such guard and produces ratios in the thousands on real data. See 4.5.
The quartet ratio distribution is computed and plotted	Python never pools these, so it cannot see when its own test statistic has degenerated, and offers no help choosing the quartet K.

9. Outputs

9.1 The drop file

One file per tree, listing taxa nominated for removal, prefixed by the rule that nominated them: `I:` long subtree, `S:` named clade stem, `T:` terminal, `Q:` quartet. Taxon names only - this file is consumed to prune alignments, so it carries taxon names and nothing else.

9.2 Clade deletion figures

Two figures, with the matching tables:

- `clade_deletion_frequencies.png` and `clade_deletion_counts.tsv` - every named clade lost to either rule.
- `clade_deletion_by_internal_rule.png` and its table - clades lost to the internal-branch rule alone.

Both use the same axes and the same stacked split: blue where the clade was removed as itself, orange where it went as an alias, sharing a branch with another clade. Comparing the pair answers a specific question: where a clade's bars match in both, the stem rule contributed nothing that the topology-driven internal rule had not already caught. Where the combined bar is tall and the internal-only bar is short, that clade is being removed only because it was named. That is the evidence for whether the named-clade rule is needed at all.

> *A clade counts as lost to the internal rule when its fragment is wholly contained in a removed subtree. A clade that loses most but not all of its taxa is not counted, so the comparison is conservative about how much the internal rule achieves.*

9.3 Coloured trees

One NEXUS file per tree. Later colours overwrite earlier ones, so the blocks run from weakest to strongest claim: named clade first, then long subtree, so the internal-branch rule takes precedence over the clade rule. A long internal branch is measured from the tree itself, whereas whether a named clade is long carries a judgement. Terminal and quartet colours follow and mark single taxa. The colours are a visual guide; the drop file is the record.

9.4 Coverage reporting

Each cross-gene rule now reports what it could not evaluate: taxa and clades below the minimum observation count, and per tree, the number of taxa for which no valid quartet could be built after masking. All are retained; the point is that a low figure signals masking that is too aggressive, or neighbourhoods with no usable scale.

Clade names are now passed into `find_lca`, so a clade that is not monophyletic in a tree is named in the message rather than reported as "unknown". The duplicate warning from inside that function is suppressed where the caller already reports the same event by name, so one non-monophyletic clade now produces one message instead of three.

10. Running it

10.1 Arguments

Flag	Meaning
-e	tree file extension
-o	extension for the output list of taxa
-c	clade definition file (required: the stem rule runs on every tree)
-x	clades excluded from quartet tests (optional)
-t	file of four K triggers: terminals, internals, stems, quartets
-s	file of three safeguard quantiles: terminals and quartets, internals, stems
-p	minimum number of stacked long internal branches (required, no default)
-r	side asymmetry ratio (required, must exceed 1.0)

10.2 Input files

Trigger file - one row of four numbers, fractional values permitted, in the order terminals, internals, stems, quartets.

```
3 3 3 3          # or, for example:  3.45 3 2.75 3
```

Safeguard file - one row of *three* quantiles. The first serves both the terminal and the quartet rule, as one value in Python serves both.

```
0.95 0.95 0.95
```

Both are length-checked at start-up. The clade files are whitespace-separated, with the clade name first - note this differs from the Python prototype, which uses a colon after the name.

10.3 Choosing the parameters

`-p` has no default and must be set deliberately: a value of 1 means any single long internal branch triggers removal, which is very liberal; 2 or 3 is normally more sensible. `-r` at 1.2 means one side must be at least 20 per cent deeper than the other; higher values demand a starker contrast and remove fewer clades.

> *The statistics script plots the distributions with alternative K values marked, and computes its global constants the same way the identifier does, so the lines drawn correspond to thresholds that would actually be applied.*

11. Library reorganisation

TreeStats had grown to 1193 lines, of which about 856 across 20 functions existed only to serve long-branch removal - 87 per cent of the file. Since the three libraries are shared with other projects, the long-branch code was moved out.

File	Before	After
TreeStats	1193 lines, 27 functions	139 lines, 6 functions
TreeUtils	924 lines, 25 functions	843 lines, 24 functions
long_branch_rules.jl	-	1164 lines, 23 functions

TreeStats now holds only tree-comparison metrics: the three Robinson-Foulds variants, Estabrook quartet similarity, patristic distances and path-length distance. Nothing in it reads a file or takes a threshold.

TreeUtils lost three functions that were rules rather than tree operations - they take a trigger or operate on a chain of long branches - and gained two that were misplaced: `path_distance`, which belongs beside its unweighted twin, and `all_split_leaf_sets`, which is split enumeration.

The rules now live in `long_branch_rules.jl`, included by both scripts. Code and comments were moved verbatim, in whole blocks, so the working notes in the source are preserved.

11.1 Verification

28 plus 25 function blocks before; 6 plus 24 plus 23 after - 53 either way. No function is defined in two files, every exported name resolves to a definition in its own file, and the splitter asserted a byte-for-byte reassembly of each original before writing anything.

12. Open questions

Two matters were left deliberately unresolved, both to be settled against real output rather than in the abstract.

12.1 Is the named-clade rule needed?

The pair of clade-deletion figures exists to answer this. If the internal-branch rule catches essentially the same clades, the stem rule can be dropped, and with it the requirement to supply clade definitions at all. That would remove the most subjective input the pipeline takes.

12.2 Residual extreme quartet ratios

Two cases survive the guard, both ctenophores in one gene tree, at ratios of 1281 and 1301 against a 99th percentile of 3.04. Their near-identical values indicate a shared anchor and comparator set, so this is one neighbourhood producing two numbers rather than two independent findings, and the values were unchanged by every guard tried - meaning those comparators have genuine branch lengths above the floor.

A ratio four hundred times the 99th percentile is not a credible biological statement, so the likely cause is comparators with small but genuinely estimated branches - a fixed cutoff catches the floored ones and misses these. The candidate remedy is a threshold relative to the dataset's branch-length scale, or a gate on the local median itself. A diagnostic has been added to the debug block to report the components of any ratio above 100, so the choice can be made on evidence rather than inference.

12.3 Fragmented clades

The fragment used is the largest monophyletic subset. Where a clade splits roughly in half, which half counts as the clade is close to arbitrary, and the treatment of the two halves will differ across genes. The safeguard still has to be cleared before anything is removed, so this does not open a hole, but it is worth watching in badly fragmented clades.

> ***A note on the numbers.*** *Several of these changes move thresholds, and not all in the same direction. Counting every observation lowers medians and quantiles, making the rules stricter. Medians of per-unit medians shift constants by an amount that depends on the sampling pattern. The minimum-observations gate and the new asymmetry conditions both reduce removals. Any parameter tuned against the previous behaviour should be revisited.*